



**Tecnicatura Universitaria en Programación**

## **Metodología de Sistemas II**

**Trabajo Práctico Final: Semana 2**

**Docente: Ing. Prof. Laurent Silvina.**

### **Integrantes:**

- **Espinosa Ezequiel**
- **Maldonado Carlos**
- **Ocampo Iván**
- **Sanchez Mauricio**

## Identificación de problema recurrente

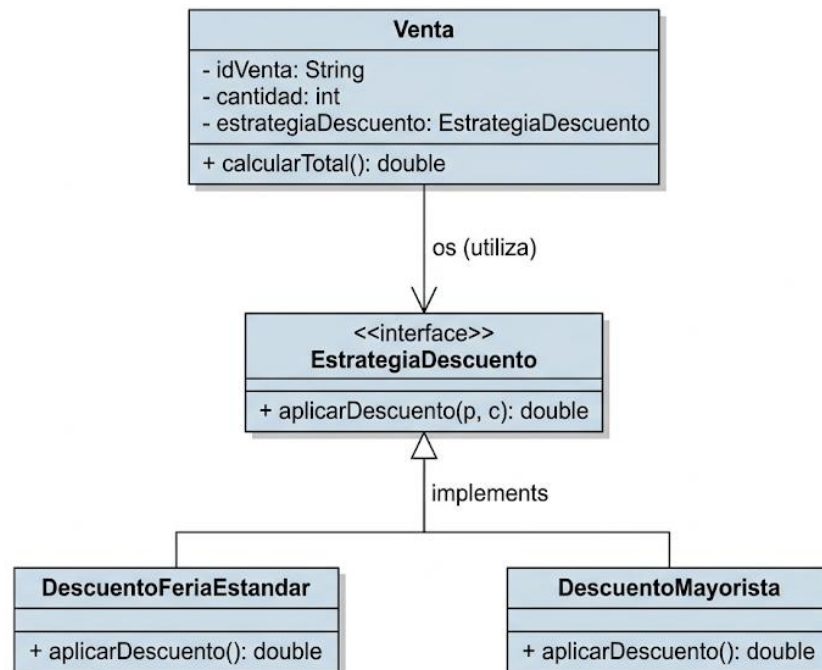
En el archivo Venta.java, el método calcularTotal() depende de una función privada llamada aplicarDescuentos(double subtotal). Esta función contiene múltiples estructuras condicionales (if) para aplicar reglas de negocio específicas (descuento por cantidad mayor a 10 y descuento por monto superior a \$5000).

Este diseño introduce un problema recurrente de **Rigidez** y **Falta de Extensibilidad**:

- **Violación del Principio de Abierto/Cerrado (OCP):** Cada vez que la organización de la feria decida añadir una nueva política de descuentos (por ejemplo, un descuento fijo por el día de la madre, promociones de fin de temporada, o tarifas diferenciales para compras mayoristas), será obligatorio abrir el archivo Venta.java y modificar o anidar más bloques condicionales.
- **Falta de cohesión:** La clase Venta asume dos responsabilidades, registrar los datos de la transacción comercial y conocer los algoritmos matemáticos exactos de las promociones vigentes.

**Patrón de diseño elegido:** Strategy (Estrategia). Este patrón de comportamiento permite definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables en tiempo de ejecución, aislando la lógica de negocio variable de la clase que la utiliza.

## Diagrama UML luego de aplicar el patrón



## Justificación Técnica

Inicialmente, se evaluó la posibilidad de crear subclases de Venta (como VentaEstandar o VentaMayorista) para sobrecribir la lógica de los cálculos. Sin embargo, este enfoque fue desestimado por su excesiva rigidez. Dado que la herencia define el comportamiento en tiempo de compilación, resulta imposible que un objeto altere su naturaleza dinámicamente durante la ejecución (por ejemplo, transicionar de una venta estándar a una mayorista al alcanzar cierto volumen de artículos). Además, si en el futuro surgieran nuevas combinaciones (por ejemplo, mezclar tipos de clientes con métodos de pago especiales), la herencia nos obligaría a crear una subclase nueva para cada combinación posible (VentaEstandarEfectivo, VentaEstandarCuotas, VentaMayoristaEfectivo, VentaMayoristaCuotas), volviendo el código más complejo. Usar el patrón **Strategy** evita esto porque se tiene una sola clase Venta a la que simplemente se le "enchufa" las piezas (estrategias) que necesitas en ese momento, manteniendo una cantidad moderada de archivos.